

Exploratory Data Analysis

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [5]: sns.set_theme(style = 'whitegrid')
plt.rcParams['figure.figsize'] = (9,5)
plt.rcParams['axes.titlesize'] = 13
plt.rcParams['axes.titleweight'] = 'bold'

DATA_PATH = 'Dataset.xlsx'

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 150)
```

1. Load Data & Overview

```
In [6]: df = pd.read_excel(DATA_PATH)
df
```

Out[6]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Pay
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	
...	...	...	...	...	...	...	...	...
1195	ORD201195	2024-06-20	C21126	Desk	1	107.04	392 Main St	
1196	ORD201196	2024-03-04	C20095	Monitor	2	662.53	778 Main St	
1197	ORD201197	2023-07-13	C79674	Tablet	2	436.84	275 Main St	
1198	ORD201198	2024-08-22	C64753	Chair	4	262.52	509 Main St	
1199	ORD201199	2023-06-11	C57502	Tablet	4	560.58	201 Main St	

1200 rows × 14 columns



In [8]:

```
# df.shape
print(f"Shape: {df.shape[0]} rows x {df.shape[1]} columns")
```

Shape: 1200 rows x 14 columns

In [9]:

```
df.columns
```

Out[9]: Index(['OrderID', 'Date', 'CustomerID', 'Product', 'Quantity', 'UnitPrice', 'ShippingAddress', 'PaymentMethod', 'OrderStatus', 'TrackingNumber', 'ItemsInCart', 'CouponCode', 'ReferralSource', 'TotalPrice'], dtype='object')

In [10]:

```
df.dtypes
```

```
Out[10]: OrderID          object
         Date            datetime64[ns]
         CustomerID      object
         Product         object
         Quantity        int64
         UnitPrice       float64
         ShippingAddress object
         PaymentMethod    object
         OrderStatus      object
         TrackingNumber   object
         ItemsInCart      int64
         CouponCode       object
         ReferralSource   object
         TotalPrice       float64
         dtype: object
```

```
In [11]: missing = df.isnull().sum()
         missing_pct = (missing / len(df) * 100).round(2)
         pd.DataFrame({'missing_count': missing, 'missing_pct': missing_pct})
```

```
Out[11]:
```

	missing_count	missing_pct
OrderID	0	0.00
Date	0	0.00
CustomerID	0	0.00
Product	0	0.00
Quantity	0	0.00
UnitPrice	0	0.00
ShippingAddress	0	0.00
PaymentMethod	0	0.00
OrderStatus	0	0.00
TrackingNumber	0	0.00
ItemsInCart	0	0.00
CouponCode	309	25.75
ReferralSource	0	0.00
TotalPrice	0	0.00

```
In [70]: df['CouponCode'] = df['CouponCode'].fillna('No Coupon')
```

```
In [72]: df.head(20)
```

Out[72]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Payment
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	
5	ORD200005	2023-10-23	C37249	Phone	2	245.86	934 Main St	
6	ORD200006	2025-06-17	C83492	Laptop	1	664.42	986 Main St	
7	ORD200007	2023-05-12	C41460	Monitor	5	149.55	706 Main St	
8	ORD200008	2025-04-02	C26817	Phone	2	134.28	904 Main St	
9	ORD200009	2023-11-21	C31946	Desk	4	509.38	102 Main St	
10	ORD200010	2023-12-29	C43443	Tablet	5	625.97	333 Main St	
11	ORD200011	2024-02-17	C93861	Monitor	3	49.14	831 Main St	
12	ORD200012	2024-10-15	C38785	Monitor	2	180.50	179 Main St	
13	ORD200013	2023-08-30	C88348	Laptop	3	201.49	903 Main St	
14	ORD200014	2023-03-27	C98474	Tablet	2	393.33	980 Main St	
15	ORD200015	2023-07-17	C39416	Printer	1	473.96	942 Main St	
16	ORD200016	2023-01-24	C28976	Printer	2	533.81	914 Main St	
17	ORD200017	2024-03-02	C60672	Tablet	1	423.40	382 Main St	
18	ORD200018	2025-03-26	C84329	Desk	1	431.44	891 Main St	

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Payme
19	ORD200019	2023-01-23	C14552	Monitor	5	224.00	273 Main St	

```
In [13]: print(f'Fully Duplicated Rows: {df.duplicated().sum()}')
```

Fully Duplicated Rows: 0

```
In [14]: print(f"Duplicated OrderIDs: {df['OrderID'].duplicated().sum()}")
```

Duplicated OrderIDs: 0

```
In [15]: print(f"Duplicated Tracking Numbers: {df['TrackingNumber'].duplicated().sum()}")
```

Duplicated Tracking Numbers: 0

```
In [16]: print('Returning Customers')
print(f"Duplicated CustomerIDs: {df['CustomerID'].duplicated().sum()}")
```

Returning Customers

Duplicated CustomerIDs: 11

2. Data Quality Checking

- Check whether the `TotalPrice` column calculated by `Quantity` x `UnitPrice`

```
In [17]: total_price = (df['Quantity'] * df['UnitPrice']).round(2)
# total_price
matching = (total_price != df['TotalPrice'].round(2))
if matching.sum() == 0:
    print('Total Price Calculated Correcly')
else:
    print('Invalid!')
```

Total Price Calculated Correcly

- Check whether `Quantity` exceed `ItemsInCart`

```
In [18]: qty_cart = (df['Quantity'] > df['ItemsInCart']).sum()
print('Number of orders where quantity exceed items in card is', qty_cart)
```

Number of orders where quantity exceed items in card is 0

- Check whether there exist non-positive values in numeric columns

```
In [19]: neg_or_zero = ((df["Quantity"] <= 0) | (df["UnitPrice"] <= 0) | (df["TotalPrice"] <= 0))
print(f"Rows with non-positive Quantity/UnitPrice/TotalPrice: {neg_or_zero.sum()}")
```

Rows with non-positive Quantity/UnitPrice/TotalPrice: 0

- Date range

```
In [20]: print(f"\nDate range: {df['Date'].min().date()} to {df['Date'].max().date()}")
```

Date range: 2023-01-01 to 2025-06-30

- Percentage of missing values in `CouponCode`

```
In [21]: print(f"Percentage of orders not used coupon code: {df['CouponCode'].isnull().mean()}")
```

Percentage of orders not used coupon code: 25.8%

3. Univariate Analysis

3.1. Univariate Analysis for Numerical Columns

```
In [22]: numeric_columns = ['Quantity', 'UnitPrice', 'ItemsInCart', 'TotalPrice']
df[numeric_columns].describe().round(2)
```

```
Out[22]:
```

	Quantity	UnitPrice	ItemsInCart	TotalPrice
count	1200.00	1200.00	1200.00	1200.00
mean	2.95	356.41	5.48	1053.97
std	1.41	197.18	2.28	819.86
min	1.00	11.39	1.00	11.39
25%	2.00	186.06	4.00	410.52
50%	3.00	364.21	5.00	823.62
75%	4.00	521.57	7.00	1578.48
max	5.00	699.93	10.00	3456.40

```
In [23]: print("Skewness:")
df[numeric_columns].skew()
```

Skewness:

```
Out[23]: Quantity      0.027922
UnitPrice    -0.026512
ItemsInCart   0.000877
TotalPrice    0.891359
dtype: float64
```

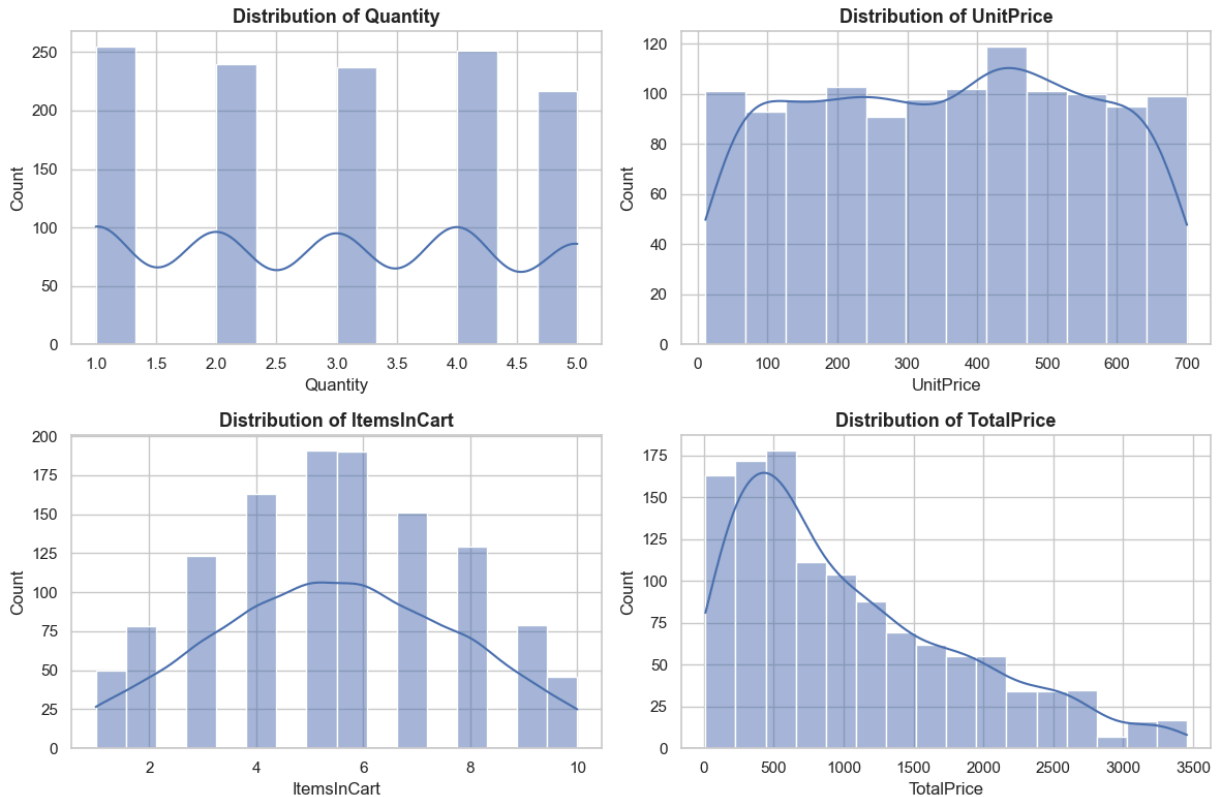
- TotalPrice positively skewed.

```
In [35]: fig, axes = plt.subplots(2, 2, figsize=(12, 8)) # gives 4 grids
```

```

for sub_plot, col in zip(axes.flat, numeric_columns):
    sns.histplot(df[col],
                 kde=True,
                 ax=sub_plot,
                 color="#4C72B0")
    sub_plot.set_title(f"Distribution of {col}")
fig.tight_layout()
plt.show()

```



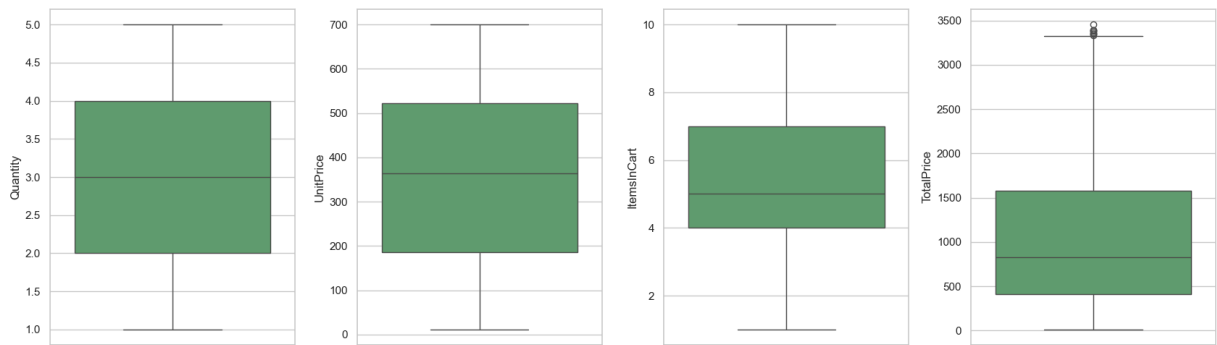
- Quantity is evenly distributed across values 1-5.
- UnitPrice shows a wide range of product prices (20 - 700) with no dominant price level.
- ItemsInCart is centered around 5-6 items, indicating a typical cart size of approximately 5-6 products.
- TotalPrice is positively skewed, suggesting most orders have low-to-moderate values while a small number of orders contributes very high sales amounts.

```

In [53]: fig, axes = plt.subplots(1, 4, figsize = (17, 5))

for sub_plot, col in zip(axes, numeric_columns):
    sns.boxplot(y = df[col],
                ax = sub_plot,
                color = "#55A868")
fig.tight_layout()
plt.show()

```



- Quantity, UnitPrice, and ItemsInCart do not contain significant outliers and exhibit relatively balanced distributions.
- TotalPrice shows positively skewed distribution with several high-value outliers.

```
In [200... print("Outlier counts (1.5x IQR rule):")
print()
for col in numeric_columns:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f" {col}: {n_out} outliers (bounds: {lower:.2f} to {upper:.2f})")
```

Outlier counts (1.5x IQR rule):

Quantity: 0 outliers (bounds: -1.00 to 7.00)
 UnitPrice: 0 outliers (bounds: -317.20 to 1024.83)
 ItemsInCart: 0 outliers (bounds: -0.50 to 11.50)
 TotalPrice: 8 outliers (bounds: -1341.41 to 3330.41)

3.2. Univariate Analysis for Categorical Columns

```
In [54]: df.head()
```

```
Out[54]:
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Paymer
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	I
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	C
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	I
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	


```
In [69]: categorical_cols = ['Product', 'PaymentMethod', 'OrderStatus', 'CouponCode', 'Refer

for col in categorical_cols:
    print(df[col].value_counts())
    print('_____')
```

```
Product
Printer    181
Tablet     179
Chair      178
Laptop     173
Desk       170
Monitor    163
Phone      156
Name: count, dtype: int64
```

```
PaymentMethod
Online      258
Cash        246
Credit Card 234
Debit Card  232
Gift Card   230
Name: count, dtype: int64
```

```
OrderStatus
Cancelled   250
Returned   247
Pending     237
Shipped     235
Delivered   231
Name: count, dtype: int64
```

```
CouponCode
FREESHIP    313
No Coupon   309
WINTER15    292
SAVE10      286
Name: count, dtype: int64
```

```
ReferralSource
Instagram    259
Email        250
Google       241
Facebook     228
Referral     222
Name: count, dtype: int64
```

```
In [88]: fig, axes = plt.subplots(3, 2, figsize=(12, 14))

for sub_plot, col in zip(axes.flat, categorical_cols):

    order = df[col].value_counts().index

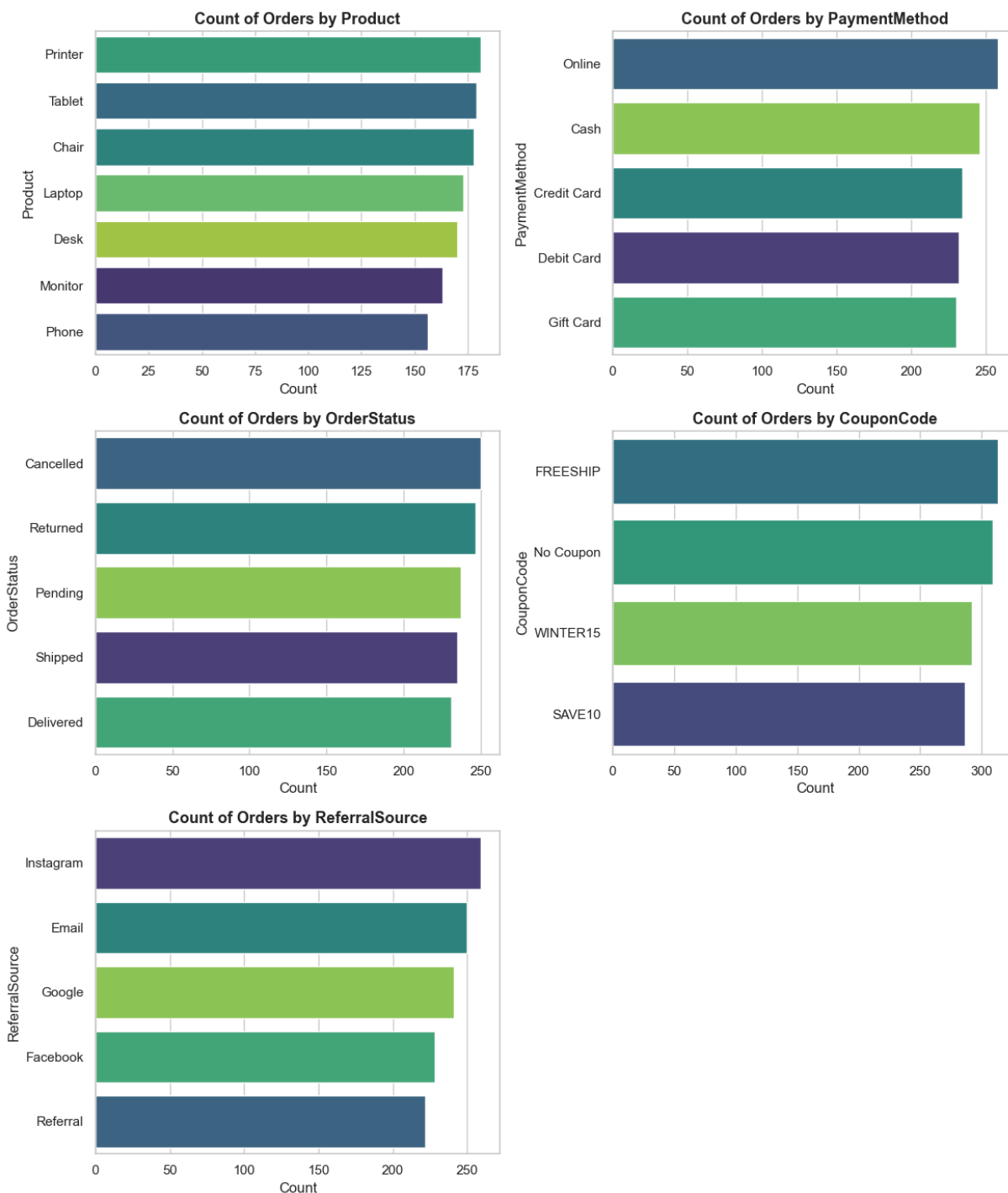
    sns.countplot(
        y=df[col],
```

```

        order=order,
        hue=df[col],
        ax=sub_plot,
        palette="viridis",
        legend=False
    )
    sub_plot.set_title(f"Count of Orders by {col}")
    sub_plot.set_xlabel("Count")

axes.flat[-1].axis("off") # hide the unused plot
fig.tight_layout()
plt.show()

```



4. Time-Based Analysis

```
In [108... df["YearMonth"] = df["Date"].dt.to_period("M")

monthly = df.groupby("YearMonth").agg(
    orders=("OrderID", "count"),
    revenue=("TotalPrice", "sum"),
    avg_order_value=("TotalPrice", "mean"), # Average Order Value = Total Revenue /
).reset_index()

monthly["YearMonth"] = monthly["YearMonth"].astype(str)
# monthly.dtypes
monthly
```

Out[108...

	YearMonth	orders	revenue	avg_order_value
0	2023-01	47	56685.75	1206.079787
1	2023-02	37	40117.66	1084.261081
2	2023-03	43	48609.37	1130.450465
3	2023-04	31	27751.71	895.216452
4	2023-05	49	63836.84	1302.792653
5	2023-06	45	49500.19	1100.004222
6	2023-07	44	42820.66	973.196818
7	2023-08	51	54352.14	1065.728235
8	2023-09	29	29526.67	1018.161034
9	2023-10	47	52607.85	1119.315957
10	2023-11	41	43079.67	1050.723659
11	2023-12	46	43754.73	951.189783
12	2024-01	32	38528.08	1204.002500
13	2024-02	32	36909.57	1153.424062
14	2024-03	36	36030.90	1000.858333
15	2024-04	50	49613.14	992.262800
16	2024-05	34	27909.11	820.856176
17	2024-06	53	68068.54	1284.312075
18	2024-07	43	42963.98	999.162326
19	2024-08	28	31991.07	1142.538214
20	2024-09	44	39794.98	904.431364
21	2024-10	31	37226.97	1200.870000
22	2024-11	35	32413.76	926.107429
23	2024-12	41	38785.77	945.994390
24	2025-01	27	29099.40	1077.755556
25	2025-02	37	35317.55	954.528378
26	2025-03	49	39200.66	800.013469
27	2025-04	32	31821.20	994.412500
28	2025-05	37	43396.64	1172.882162
29	2025-06	49	53047.40	1082.600000

```
In [142... metrics = ['orders', 'revenue', 'avg_order_value']
colors = ['#4C72B0', '#C44E52', '#55A868']

fig, axes = plt.subplots(3, 1, figsize=(12, 15))

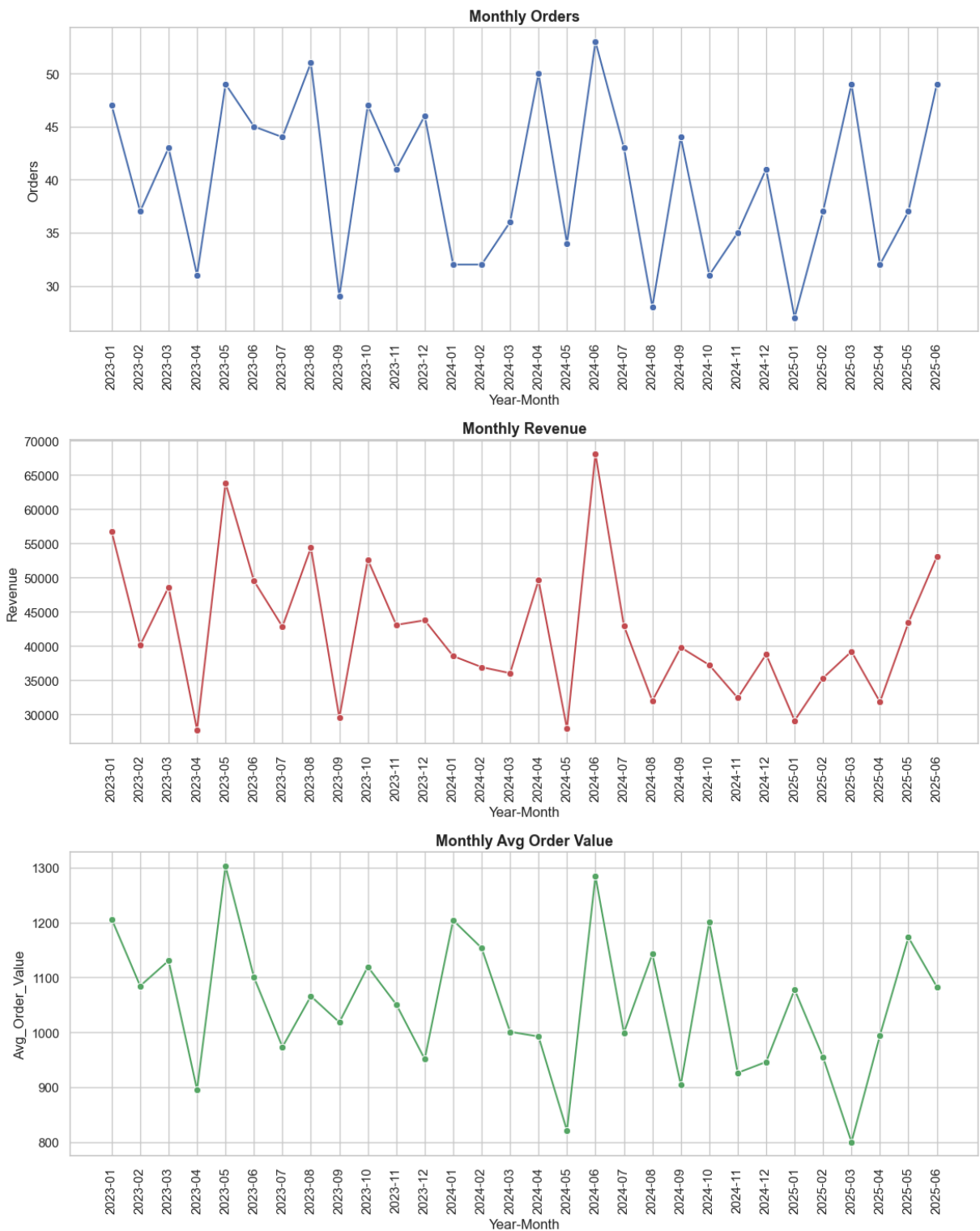
for ax, metric, color in zip(axes, metrics, colors):

    sns.lineplot(
        data=monthly,
        x='YearMonth',
        y=metric,
        marker='o',
        ax=ax,
        color = color
    )

    ax.set_title(f'Monthly {metric.replace("_", " ").title()}')
    ax.set_xlabel('Year-Month')
    ax.set_ylabel(metric.title())

    ax.tick_params(axis='x', rotation=90)

plt.tight_layout()
plt.show()
```



In [141...]

```
df['WeekDay'] = df['Date'].dt.day_name()
week_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
weekly_count = df['WeekDay'].value_counts().reindex(week_order)

fig, ax = plt.subplots()

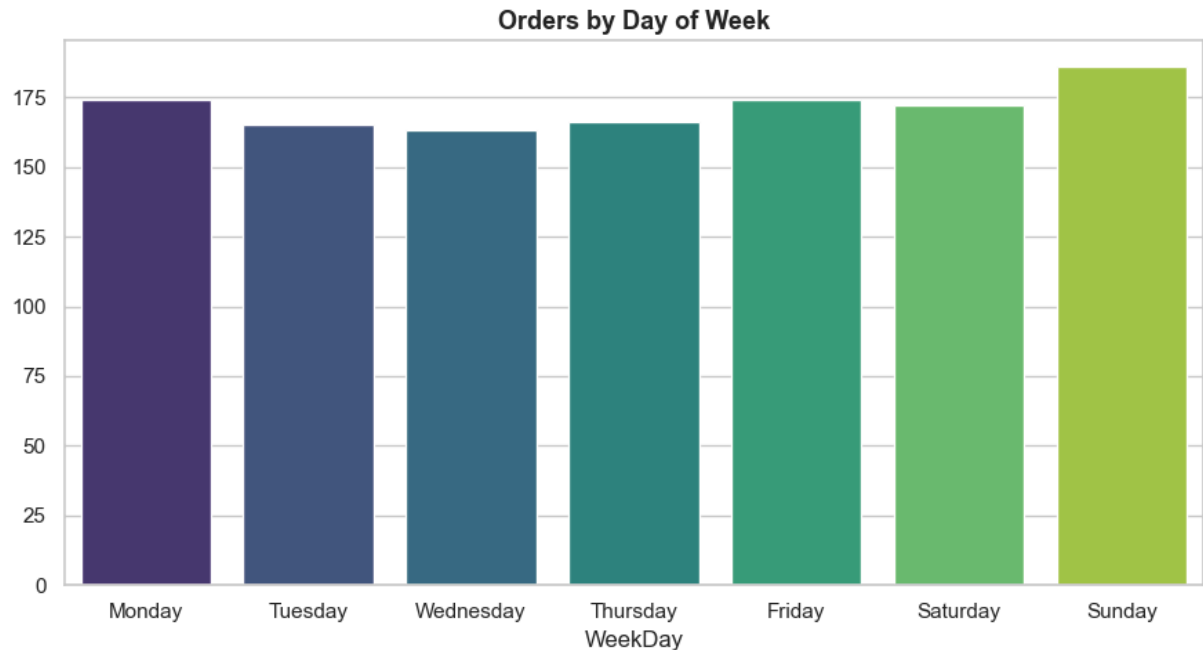
sns.barplot(
    x = weekly_count.index,
    y = weekly_count.values,
    hue = weekly_count.index,
```

```

ax = ax,
palette = 'viridis'
)

ax.set_title("Orders by Day of Week")
fig.tight_layout()
plt.show()

```



5. Relationships Between Variables

```

In [148...] df.groupby("Product")["TotalPrice"]. \
              agg(["mean", "median", "sum", "count"]).round(2) \
              .sort_values("sum", ascending=False)

```

```

Out[148...]

```

	mean	median	sum	count
Product				
Chair	1098.99	928.58	195620.11	178
Printer	1080.73	857.40	195612.61	181
Laptop	1110.56	915.64	192126.56	173
Tablet	1042.28	786.66	186568.95	179
Monitor	1077.62	853.40	175651.41	163
Desk	985.06	784.39	167459.93	170
Phone	972.58	691.71	151722.39	156

mean = average order value

- Laptop has the highest average order value.

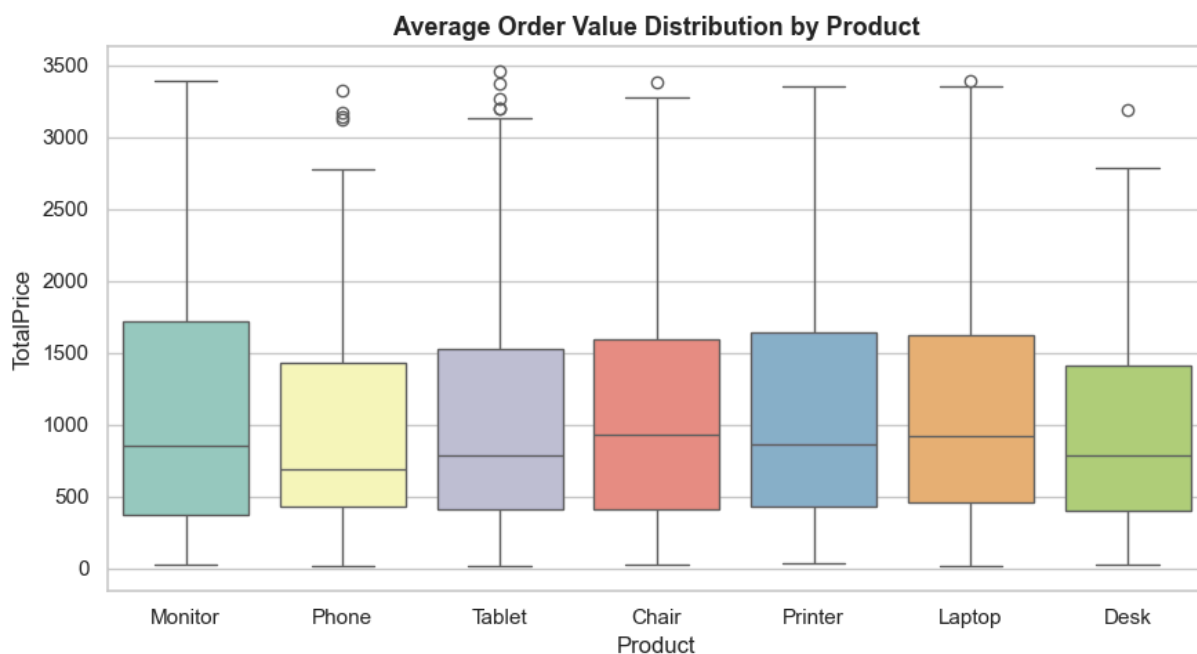
- Printer is the most frequently ordered product.

In [162...

```
fig, ax = plt.subplots()

sns.boxplot(data=df,
            x="Product",
            y="TotalPrice",
            hue="Product",
            ax=ax,
            palette="Set3",
            legend=False)

ax.set_title("Average Order Value Distribution by Product")
fig.tight_layout()
plt.show()
```



- Product-level boxplots show that Chair and Laptop orders have the highest typical spending levels, while Phone orders have the lowest.
- All product categories exhibit right-skewed distributions with several high-value outliers, indicating occasional large purchases.

In [153...

```
df.groupby("ReferralSource")["TotalPrice"]. \
    agg(["mean", "median", "count"]). \
    sort_values("mean", ascending=False)
```


Out[153...

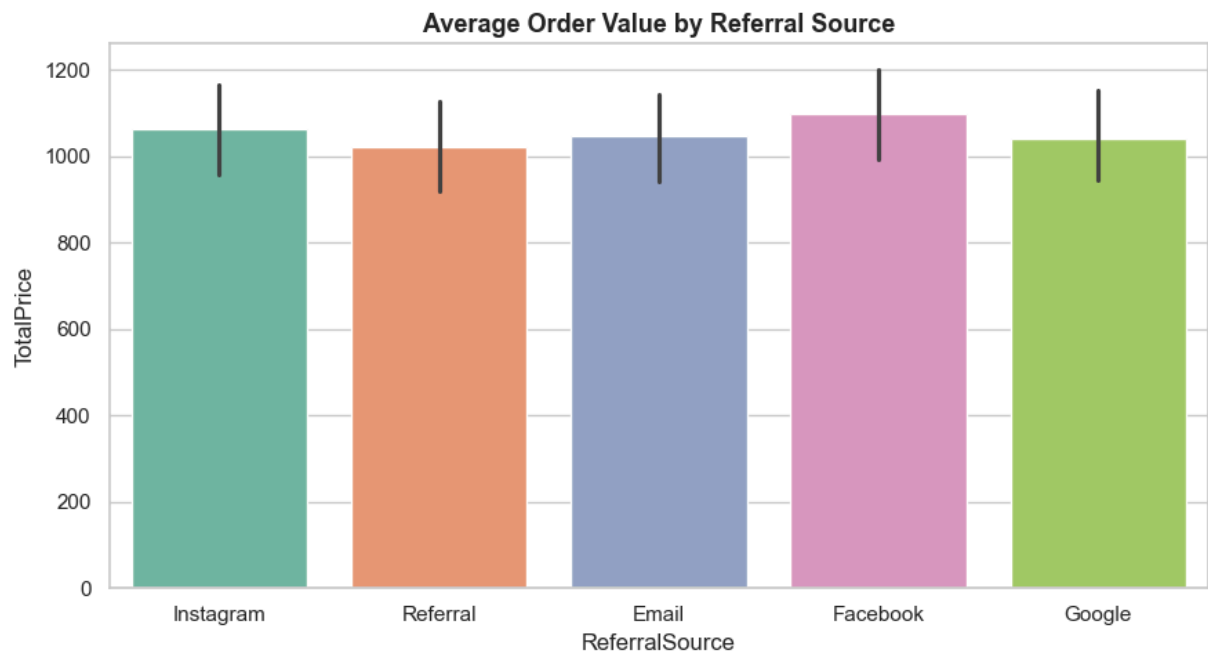
	mean	median	count
ReferralSource			
Facebook	1098.293421	911.515	228
Instagram	1062.878185	856.180	259
Email	1047.234200	758.190	250
Google	1039.176266	839.960	241
Referral	1021.691802	734.575	222

In [163...

```
fig, ax = plt.subplots()

sns.barplot(data=df,
            x="ReferralSource",
            y="TotalPrice",
            hue="ReferralSource",
            ax=ax,
            palette="Set2")

ax.set_title("Average Order Value by Referral Source")
fig.tight_layout()
plt.show()
```



In [190...

```
df2 = (pd.crosstab(df["PaymentMethod"], df["OrderStatus"], normalize="index") * 100)
df2
```

Out[190...

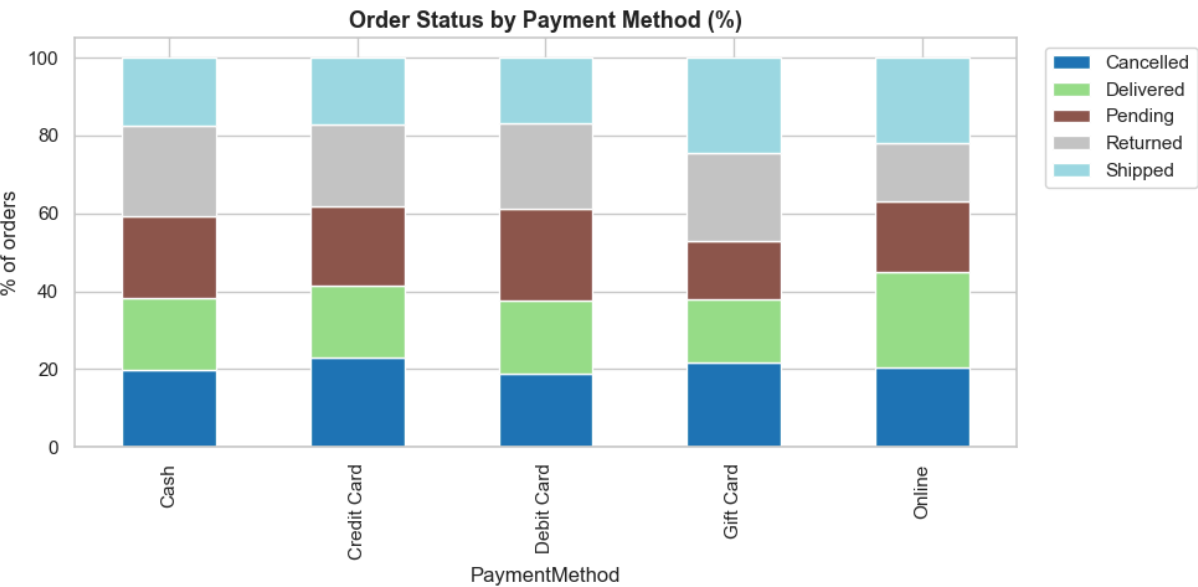
OrderStatus	Cancelled	Delivered	Pending	Returned	Shipped
PaymentMethod					
Cash	19.92	18.29	21.14	23.17	17.48
Credit Card	23.08	18.38	20.51	20.94	17.09
Debit Card	18.97	18.53	23.71	21.98	16.81
Gift Card	21.74	16.09	15.22	22.61	24.35
Online	20.54	24.42	18.22	14.73	22.09

In [192...

```
fig, ax = plt.subplots(figsize=(10, 5))

df2.plot(kind="bar",
          stacked=True,
          ax=ax,
          colormap="tab20")

ax.set_title("Order Status by Payment Method (%)")
ax.set_ylabel("% of orders")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
fig.tight_layout()
plt.show()
```



In [196...

```
df3 = (pd.crosstab(df["Product"], df["OrderStatus"], normalize="index") * 100).round
df3
```

Out[196...

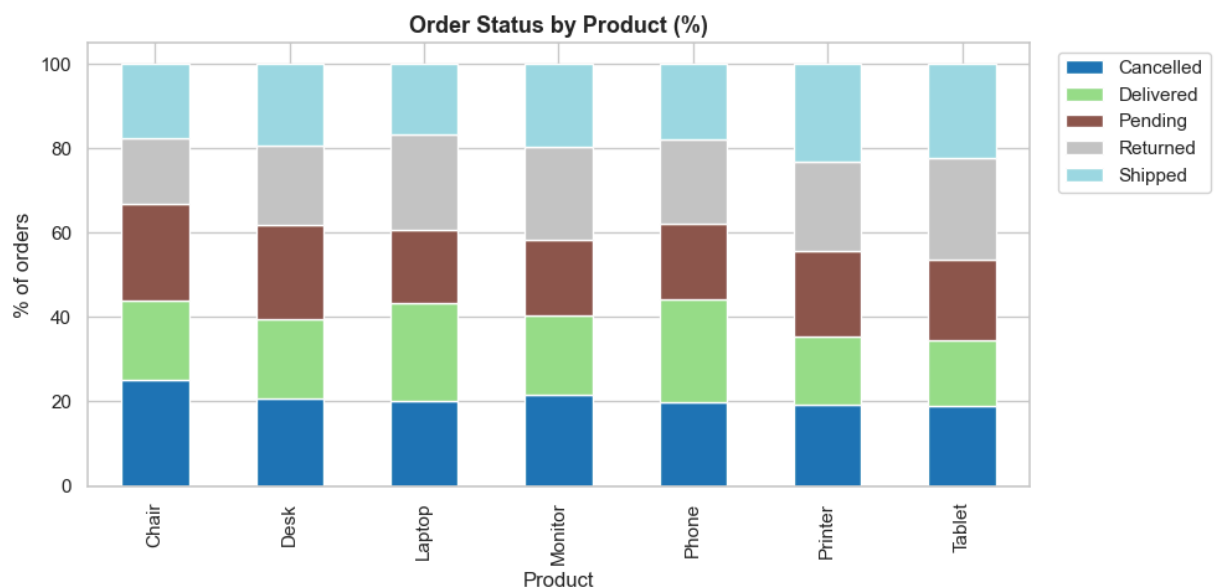
OrderStatus	Cancelled	Delivered	Pending	Returned	Shipped
Product					
Chair	25.28	18.54	23.03	15.73	17.42
Desk	20.59	18.82	22.35	18.82	19.41
Laptop	20.23	23.12	17.34	22.54	16.76
Monitor	21.47	19.02	17.79	22.09	19.63
Phone	19.87	24.36	17.95	19.87	17.95
Printer	19.34	16.02	20.44	20.99	23.20
Tablet	18.99	15.64	18.99	24.02	22.35

In [197...

```
fig, ax = plt.subplots(figsize=(10, 5))

df3.plot(kind="bar",
          stacked=True,
          ax=ax,
          colormap="tab20")

ax.set_title("Order Status by Product (%)")
ax.set_ylabel("% of orders")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
fig.tight_layout()
plt.show()
```



In [194...

```
df.groupby("CouponCode")["TotalPrice"]. \
  agg(["mean", "median", "count"]).round(2). \
  sort_values("mean", ascending=False)
```

Out[194...

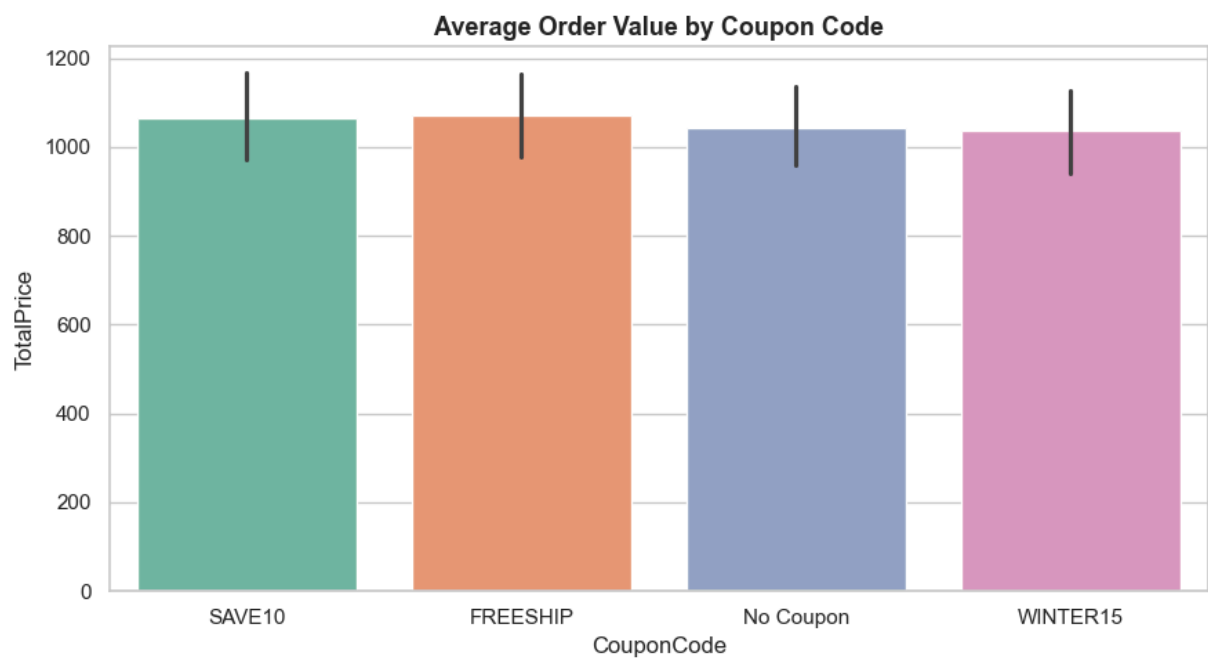
	mean	median	count
CouponCode			
FREESHIP	1070.41	793.40	313
SAVE10	1065.87	823.06	286
No Coupon	1043.37	834.12	309
WINTER15	1035.90	838.71	292

In [168...

```
fig, ax = plt.subplots()

sns.barplot(data=df,
            x="CouponCode",
            y="TotalPrice",
            hue="CouponCode",
            ax=ax,
            palette="Set2")

ax.set_title("Average Order Value by Coupon Code")
fig.tight_layout()
plt.show()
```



In [172...

```
(pd.crosstab(df["Product"], df["OrderStatus"], normalize="index") * 100).round(2)
```

Out[172...

OrderStatus	Cancelled	Delivered	Pending	Returned	Shipped
Product					
Chair	25.28	18.54	23.03	15.73	17.42
Desk	20.59	18.82	22.35	18.82	19.41
Laptop	20.23	23.12	17.34	22.54	16.76
Monitor	21.47	19.02	17.79	22.09	19.63
Phone	19.87	24.36	17.95	19.87	17.95
Printer	19.34	16.02	20.44	20.99	23.20
Tablet	18.99	15.64	18.99	24.02	22.35

6. Correlation Between Numeric Variables

In [175...

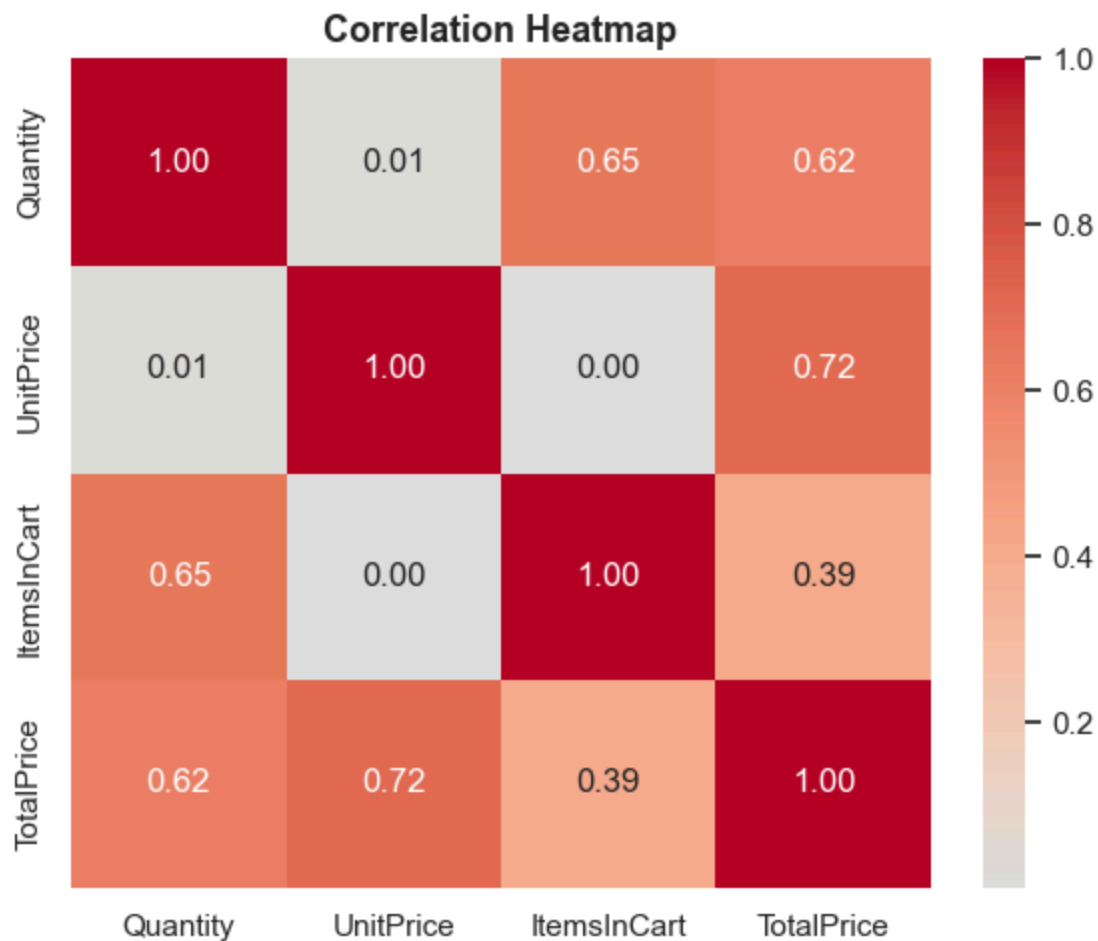
```
corr = df[numeric_columns].corr()  
corr.round(2)
```

Out[175...

	Quantity	UnitPrice	ItemsInCart	TotalPrice
Quantity	1.00	0.01	0.65	0.62
UnitPrice	0.01	1.00	0.00	0.72
ItemsInCart	0.65	0.00	1.00	0.39
TotalPrice	0.62	0.72	0.39	1.00

In [177...

```
fig, ax = plt.subplots(figsize=(6, 5))  
  
sns.heatmap(corr,  
             annot=True,  
             cmap="coolwarm",  
             center=0,  
             ax=ax,  
             fmt=".2f")  
  
ax.set_title("Correlation Heatmap")  
fig.tight_layout()  
plt.show()
```



7. Customer-Level Summary

```
In [179... cust = df.groupby("CustomerID").agg(
    n_orders=("OrderID", "count"),
    total_spend=("TotalPrice", "sum"),
    avg_order_value=("TotalPrice", "mean"),
).sort_values("total_spend", ascending=False)

print(f"Unique customers: {df['CustomerID'].nunique()}")
print(f"Returning Customers: {(cust['n_orders'] > 1).sum()}")
cust.head(10)
```

Unique customers: 1189
Returning Customers: 11

Out[179...

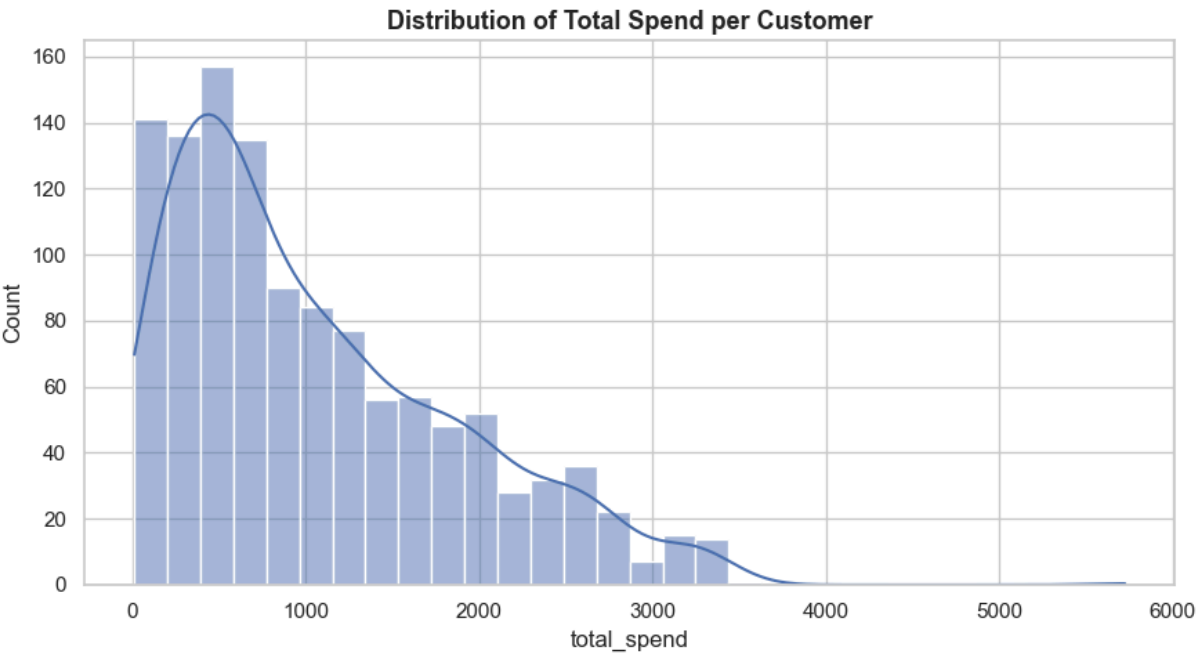
	n_orders	total_spend	avg_order_value
CustomerID			
C38840	2	5723.23	2861.615
C57276	1	3456.40	3456.400
C67260	1	3390.80	3390.800
C13877	1	3384.90	3384.900
C18404	1	3370.20	3370.200
C16775	1	3353.75	3353.750
C65986	1	3352.40	3352.400
C47778	1	3334.00	3334.000
C59183	1	3322.55	3322.550
C25276	1	3313.90	3313.900

In [180...

```
fig, ax = plt.subplots()

sns.histplot(cust["total_spend"],
             bins=30,
             color="#4C72B0",
             kde=True,
             ax=ax)

ax.set_title("Distribution of Total Spend per Customer")
fig.tight_layout()
plt.show()
```



8. Executive Summary

1. Problem Statement

Why the order cancellation is high?

2. Methodology

Checked payment method, product, channel, and coupon use against cancellations, and ran proper statistical tests.

3. Key Findings

Cancellation rate is 20.8%, but no single factor explains it, payment method, product, channel, and coupon usage all showed no statistically significant link.

4. Recommendations

Check for customer-level repeat cancellations